Arjun Patel
CS 385 Homework 2
18 October 2024
Professor Eiers

"I pledge my honor that I have abided by the Stevens Honor System" – Arjun Patel

pg 67 #4 a-e

4) Algorithm Mystery(n)
// Input: A nonnegative integer n
S ← 0
for i ← 1 to n do
    S ← S+i * i
return S

```
S = 0
for ( int i=0; i<= n ; i++ )
    S = S + i * i
return s
```

a) what does this algorithm compute?

This algorithm computes the sum of the squares for numbers starting at 1 up to and including n. For example, if n is equal to 3, $S = 1^2 + 2^2 + 3^2 = 14$ or $\sum_{i=1}^{n} i^2$

S is initialized to be 0
S = 0
1st iteration (i=1): S = S + 1 x 1
S = 0 + 1
S = 1
2nd iteration (i=2): S = S + 2 x 2
S = 1 + 4
S = 5
3rd iteration (i=3): S = S + 3 x 3
S = 5 + 9
S = 14

loop breaks since i > n and in our ex n = 3

14 is returned

b) what is its basic operation?

The basic operations involved in the program are multiplication and addition. Multiplication to calculate the square of numbers between i=1 to n, and Addition to add all the squared values up. The basic operation(s) occur in the line: S ← S+i*i
At each iteration, i is multiplied by itself from i=1 to n, and at each iteration the value of S is added to i x i

c) How many times is the basic operation executed?

The basic operation is executed n times. The algorithm does one multiplication operation for every value of n, so it would be n multiplication operations. Similarly, for each value of n, S gets added to $i \times i$.

d) What is the efficiency class of this algorithm?

The efficiency class of this algorithm is linear, namely $\Theta(n)$. For each iteration, $S \leftarrow S + i * i$ involves two operations (multiplication and addition) n times as a result of the for loop.

e) Suggest any improvement, or a better algorithm altogether, and indicate its efficiency class. If you cannot do it, try to prove that, in fact, it cannot be done.

After analyzing the algorithm, we observed that its efficiency class was linear.

However, we can transform the algorithm from linear efficiency to constant efficiency by eliminating the for loop.

```
Algorithm Mystery(n)
// Input: A nonnegative integer n
S ← 0
for i ← 1 to n do
    S ← S + i * i
return S
```

Using a mathematical formula to directly compute the sum of squares $\Rightarrow$ closed-form solution for sum of squares

$$\sum_{i=1}^{N} i^2 = \frac{N(N+1)(2N+1)}{6}$$

Using the formula and eliminating the loop the algorithm becomes

```
Algorithm Mystery(n)
// Input. A nonnegative integer n
return (n * (n+1) * (2n+1)) / 6
```

We now have a improved the algorithm from linear efficiency to constant efficiency.

pg 76 # 1 a-e

1a) $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$

Step 1: replace n by $n-1$, substitute to calculate next iteration and plug back into original (unfold 1st time)

$x(n-1) = x(n-1-1) + 5$

$x(n-1) = x(n-2) + 5$

$x(n) = x(n-1) + 5$

$= x(n-2) + 5 + 5$ ← plug back into $x(n)$ since we know what $x(n-1)$ is

Step 2: replace n by $n-2$, substitute to calculate next iteration and plug back into original (unfold 2nd time)

$x(n-2) = x(n-2-1) + 5$

$x(n-2) = x(n-3) + 5$

$x(n) = x(n-2) + 5 + 5$

$x(n) = x(n-3) + 5 + 5 + 5$ ← plug back into $x(n)$ since we know what $x(n-2)$ is

Step 3: Analyze the results from unfolding the recurrance relation, and generalize for i iterations

$x(n) = x(n-i) + 5i$

step 4: initial condition $x(1) = 0$

for $x(1) = 0$ to work, since we want to simplify $x(n-i)$ we set $n-i = 1$ and solve for n. $\Rightarrow$ $n = 1+i$

Step 5: plugging in $n = 1+i$ to our recurrance relation from Step 3,

$x(n) = x([1+i] - i) + 5i$

$= \underbrace{x(1)}_{0} + 5i$

$= 5i$ ← since we need to express in terms of n, $i = n-1$

$x(n) = 5(n-1) = 5n - 5$

b) $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$

Step 1: replace n by $n-1$, substitute to calculate next iteration and plug back into original (unfold 1st time)

$x(n-1) = 3x(n-1-1)$

$x(n-1) = 3x(n-2)$

$x(n) = 3x(n-1)$

$= 3[3x(n-2)]$ ← plug back into $x(n)$ since we know what $x(n-1)$ is

Step 2: replace n by $n-2$, Substitute to calculate next iteration and plug back into original (unfold 2nd time)

$x(n-2) = 3x(n-2-1)$

$x(n-2) = 3x(n-3)$

$x(n) = 3[3x(n-2)]$

$= 3[3[3x(n-3)]]$ ↷ plug back into $x(n)$ since we know what $x(n-2)$ is

or $3^3 x(n-3)$

Step 3: Analyze the results from unfolding the recurrance relation, and generalize for i iterations

$x(n) = 3^i \cdot x(n-i)$

Step 4: Initial condition $x(1) = 4$, use to simplify $x(n-i)$

$n - i = 1$

$n = 1 + i$

Step 5: plug in $n = 1+i$ to recurrance relation in step 3 $\quad i = n-1$

$x(n) = 3^i \cdot x([1+i] - i)$

$= 3^i \cdot \underbrace{x(1)}_{4}$ ↷ $3^i \cdot 4$ but we want in terms of n,

$\boxed{x(n) = 3^{n-1} \cdot 4}$

c) $x(n) = x(n-1) + n$ for $n > 0$, $x(0) = 0$

Step 1. replace n by $n-1$, substitute to calculate next iteration and plug back into original (unfold 1st time)

$x(n-1) = x(n-1-1) + (n-1)$

$x(n-1) = x(n-2) + (n-1)$

$x(n) = x(n-1) + n$

$= x(n-2) + (n-1) + n$ ↷ plug back into $x(n)$ since we know what $x(n-1)$ is

Step 2: replace n by $n-2$, substitute to calculate next iteration and plug back into original (unfold 2nd time)

$x(n-2) = x(n-2-1) + (n-2)$

$x(n-2) = x(n-3) + (n-2)$

$x(n) = x(n-2) + (n-1) + n$

$= x(n-3) + (n-2) + (n-1) + n$ ↷ plug back into $x(n)$ since we know $x(n-2)$

Step 3: Analyze the results from unfolding the recurrance relation, and generalize for i iterations

$x(n) = x(n-i) + (n-i+1) + (n-i+2) + \ldots + \ldots + n$

Step 4: Initial condition $x(0) = 0$, to simplify $x(n-i)$ term, set

$n - i = 0$

$n = i$

Step 5: plug in $n = i$ into recurrence relation in step 3

$x(n) = x(n-i) + (n-i+1) + (n-i+2) + \ldots + n$

$x(n) = \underbrace{x(0)}_{0} + 1 + 2 + \ldots n = \boxed{x(n) = \frac{n(n+1)}{2}}$

d) $x(n) = x\left(\frac{n}{2}\right) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)

step 0: make the initial substitution of $n = 2^k$ into $x(n)$

$x(n) = x\left(\frac{n}{2}\right) + n$

$x(2^k) = x\left(\frac{2^k}{2}\right) + 2^k$

$x(2^k) = x(2^{k-1}) + 2^k$

Step 1: replace 2^k with 2^{k-1}, subsitute to calculate next iteration and plug back into original (unfold 1st time)

$x(2^{k-1}) = x(2^{k-1-1}) + 2^{k-1}$

$x(2^{k-1}) = x(2^{k-2}) + 2^{k-1}$

$x(2^k) = x(2^{k-1}) + 2^k$

$= x(2^{k-2}) + 2^{k-1} + 2^k$ ← plug back into $x(2^k)$ since we know $x(2^{k-1})$

step 2: replace 2^k with 2^{k-2}, subsitute to calculate next iteration and plug back into original (unfold 2nd time)

$x(2^{k-2}) = x(2^{k-2-1}) + 2^{k-2}$

$x(2^{k-2}) = x(2^{k-3}) + 2^{k-2}$

$x(2^k) = x(2^{k-2}) + 2^{k-1} + 2^k$

$= x(2^{k-3}) + 2^{k-2} + 2^{k-1} + 2^k$ ← plug back into $x(2^k)$ since we know $x(2^{k-2})$

step 3: Analyze results from unfolding the recurrence relation and generalize for i iterations

$x(2^k) = x(2^{k-i}) + 2^{k-i+1} + 2^{k-i+2} + \ldots + 2^k$

step 4: Initial condition $x(1) = 1$

try to simplify $x(2^{k-i})$ term

$2^{k-i} = 1$

rewnite 1 as 2^0

$2^{k-i} = 2^0$

$\log_2$ on both sides

$k - i = 0$

$k = i$

step 5. plug $k=i$ back into recurrence relation in step 3

$x(2^k) = x(2^{k-i}) + 2^{k-i+1} + 2^{k-i+2} + \ldots + 2^k$

$= \underbrace{x(2^0)}_{0} + 2^1 + 2^2 + 2^3 + \ldots 2^k$ ⇒ geometric series

$\sum_{i=0}^{N} A^i = \frac{A^{N+1} - 1}{A - 1}$ where $A \neq 1$

$\frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1$

$n = 2^k$

$k = \lg_2(n)$

$2^{\overbrace{\lg_2(n)}^{n}} \cdot 2^1 - 1$

$2^{\lg_2(n)+1} - 1$

$= x(n) = 2n - 1$

e) $x(n) = x\left(\frac{n}{3}\right) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)

step 0: make the initial substitution of $n = 3^k$ into $x(n)$

$$x(n) = x\left(\frac{n}{3}\right) + 1$$

$$x(3^k) = x\left(\frac{3^k}{3}\right) + 1 \Rightarrow x(3^k) = x(3^{k-1}) + 1$$

step 1: replace 3^k with 3^{k-1}, substitute to calculate next iteration and plug back into original (unfold 1st time)

$$x(3^{k-1}) = x(3^{k-1-1}) + 1$$

$$x(3^{k-1}) = x(3^{k-2}) + 1$$

$$x(3^k) = x(3^{k-1}) + 1$$

$$= x(3^{k-2}) + 1 + 1$$

↩ plug back into $x(n)$ since we know $x(3^{k-1})$

step 2: replace 3^k with 3^{k-2}, substitute to calculate next iteration and plug back into original (unfold 2nd time)

$$x(3^{k-2}) = x(3^{k-2-1}) + 1$$

$$x(3^{k-2}) = x(3^{k-3}) + 1$$

$$x(3^k) = x(3^{k-2}) + 1 + 1$$

$$= x(3^{k-3}) + 1 + 1 + 1$$

step 3: Analyze results from unfolding the recurrence rel and generalize for i iterations

$$x(3^k) = x(3^{k-i}) + 1i$$

step 4. $x(1) = 1$, so to simplify $x(3^{k-i})$

$$3^{k-i} = 1$$

rewrite as $3^0 \Rightarrow 3^{k-i} = 3^0$

$$k - i = 0$$

$$k = i$$

step 5: plug $k = i$ back into step 3

$$x(3^k) = x(3^{k-i}) + 1i$$

$$x(3^k) = \underbrace{x(3^0)}_{1} + 1i$$

$$x(3^k) = 1 + 1i$$

$$x(3^k) = 1 + 1k \text{ since } i = k$$

$$n = 3^k$$

$$k = \log_3(n)$$

$$x(n) = 1 + \log_3(n)$$

pg 76-77 3 a & b

3) Consider the following recursive algorithm for computing the sum of the first n cubes. $S(n) = 1^3 + 2^3 + \dots + n^3$

```
Algorithm S(n)
// input: A positive integer n
// output: The sum of the first n cubes
if n=1 return 1
else return S(n-1) + n * n * n
```

a) Set up and solve a recurrance relation for the number of times the algorithm's basic operation is executed.

$T(n) = 1 \cdot T(n-1) + 2$, $T(1) = 0$

Amount of sub calls + work per call

Step 1. plug in $T(n-1)$ for $T(n)$ to unfold once

$T(n) = T(n-1) + 2$

$T(n-1) = T(n-1-1) + 2$

$T(n-1) = T(n-2) + 2$

$T(n) = T(n-1) + 2$

$= T(n-2) + 2 + 2$

Step 2: plug in $T(n-2)$ for $T(n)$ to unfold 2nd time

$T(n-2) = T(n-2-1) + 2$

$T(n-2) = T(n-3) + 2$

$T(n) = T(n-2) + 2 + 2$

$= T(n-3) + 2 + 2 + 2$

Step 3: generalize results

$T(n) = T(n-i) + 2i$

Step 4: Use inital condition $T(1) = 0$ to simplify $T(n-i)$ in step 3

$n - i = 1$

$n = 1 + i$

Step 5: plug in $n = 1+i$ into step 3

$T(n) = T(1+i-i) + 2i$

$T(n) = T(1) + 2i$

$T(1) = 0$

$T(n) = 2i$

but we want in terms of $n \Rightarrow i = n-1$

$T(n) = 2(n-1) = 2n-2$

b) How does this algorithm compare with the straitforward nonrecursive algorithm for computing this sum?

nonrecursive approach →

```
s = 0                    (iterative)
for i = 1 to n do
    s = s + i x i x i
return s
```

The recursive algorithm is slower than the iterative approach for the algorithm. This is because recursion takes up more memory, specifically on the stack due to the function call. The time complexity of the iterative version would be $\Theta(1)$, and the time complexity of the recursive version would be $\Theta(n)$. The iterative version would be faster and more efficient.